

AutoGraphForge: Conjecturing, Refutation and Lean Proving in One Loop

Ján Pastorek^[0000–0001–8237–1275]

Department of Applied Informatics, Comenius University in Bratislava, Slovakia
`jan.pastorek@fmph.uniba.sk`

Abstract. AutoGraphForge couples conjecturing, refuting, formalizing and proving into one loop for graph theory. Counterexample-guided rounds produced 6,522 conjectures that survive an implication-deciding novelty filter over 559 known relations, refutation against a dataset of 348,207 (extremal) graphs and counterexample search algorithms; survivors are deterministically exported to Lean 4 with Mathlib for kernel-checked provers to attempt.

Keywords: Automated conjecturing · Graph theory · Automated theorem proving · Lean · Counterexample search

Motivation. Graph theory conjecturing systems such as GRAFFITI [4] and now the recent GRAFFITI³ [3] have developed separately from automated provers and proof systems such as Lean 4 [7]. Then, there are many sophisticated search algorithms in the literature aiming to find (counter)examples to conjectures [5]. With recent advancements in AI, we aim to develop a system that joins all these automations together—automated conjecturing, refutation, and proving into one loop. AutoGraphForge is our attempt at one (code, data and logs at <https://github.com/JanPastorek/AutoGraphForge>). It works in rounds, every counterexample found is inserted into the dataset that drives the next round.

Generation of conjectures. We use GRAFFITI³ to generate conjectures over a *small* evolving dataset T carrying 59 exactly computed invariants from GRAPHCALC [2]. Crucially this T grows *only* by counterexamples to its own conjectures, so generation stays cheap.

Novelty filter as an LP feasibility test. One of the problems with such conjecturing systems has always been to separate known, proven or trivial conjectures from unknown and potentially interesting ones. We curate 559 classical, folklore and trivial relations, closed under transitive composition and linear identity substitution, each tagged with the class on which it holds. A candidate conjecture $f \leq R(g)$ —target invariant f , with R a linear combination of the remaining invariants g —is redundant if a convex combination of tabled bounds B_j on f dominates it, certified by weights $w_j \geq 0$, $\sum_j w_j = 1$, slacks $s_i, t \geq 0$ and free multipliers μ_k with, identically in g ,

$$R(g) - \sum_j w_j B_j(g) = \sum_i s_i (g_i - L_i) + \sum_k \mu_k E_k(g) + t,$$

for $g_i \geq L_i$ safe lower bounds and $E_k(g) = 0$ class identities (e.g. Gallai’s $n = \alpha + \tau$ on order, independence and vertex-cover numbers); matching coefficients reduces this to LP feasibility. This makes “is this already known?” a decidable query against a curated library of known theorems and trivial relations.

Refutation, formalization, proving. Surviving conjectures are tested against 348,207 graphs unioning the House of Graphs export [1], all connected graphs on at most nine vertices and some extremal graph families, then against graphs from several random graph models and by six automated counterexample-search algorithms chosen from [5]. Since a conjecture is already a structured object—named invariants under class predicates—the Lean 4 export is deterministic, yielding a skeleton ending in `sorry` that elaborates to a type-correct goal. The AI agent OPROVER-32B [6] then searches for proofs, kernel-checked against pinned `mathlib4` and our preamble.

Results and status. After 5 rounds of generation–refutation, 6,522 conjectures survived. We checked the 100 survivors sharp on the most graphs and with the help of AI tagged 48 as not known to be trivial or false; among these is a pair relating the annihilation number a (the largest k whose k smallest degrees sum to at most the edge count) and the edge-cover number ec that we then proved by hand—bipartite $\Rightarrow ec \leq a$, regular $\Rightarrow a \leq ec$ —via Gallai, König and Pepper’s $\alpha \leq a$ [8]. So far the proving stage has kernel-verified only trivial bounds, and scaling it to the survivors is our next step.

Acknowledgments. Computations used the PERUN supercomputer (Supercomputing Centre, TUKE, Slovakia), funded by the EU Recovery and Resilience Plan of the Slovak Republic, project No. 17I03-04-P03-00001.

Disclosure of Interests. The author has no competing interests to declare.

References

1. Coolsaet, K., D’hondt, S., Goedgebeur, J.: House of graphs 2.0: a database of interesting graphs and more. *Discrete Applied Mathematics* **325**, 97–107 (2023). <https://doi.org/10.1016/j.dam.2022.10.013>
2. Davila, R.: GraphCalc: A Python package for computing graph invariants in automated conjecturing systems. *Journal of Open Source Software* **10**(112), 8383 (2025). <https://doi.org/10.21105/joss.08383>
3. Davila, R.: Graffiti³: Compact theory libraries for automated mathematical discovery. Preprint (2026). <https://doi.org/10.13140/RG.2.2.20757.18402>
4. Fajtlowicz, S.: On conjectures of Graffiti. *Discrete Mathematics* **72**(1–3), 113–118 (1988)
5. Jooker, J.: Computer-assisted graph theory: a survey. arXiv:2508.20825 (2025)
6. M-A-P: OProver: A unified framework for agentic formal theorem proving. arXiv:2605.17283 (2026)
7. de Moura, L., Kong, S., Avigad, J., Van Doorn, F., von Raumer, J.: The Lean theorem prover (system description). In: CADE. pp. 378–388. Springer (2015)
8. Pepper, R.: Binding Independence. Ph.D. thesis, University of Houston (2004)